

WebXR Hidden Ninja Demo v2

팀 공유용 아키텍처 · 기능 · 개발 방향 정리

한 문장 요약

Android Chrome에서 WebXR/ARCore로 현실 공간의 위치와 표면을 추적하고, 실제 표면 좌표 중 하나에 반투명 가상 캐릭터를 숨긴 뒤, 사용자가 공간을 이동하면서 거리와 시야각을 이용해 찾는 AR 숨바꼭질 프로토타입입니다.

- 핵심 검증: 브라우저에서도 ARCore 기반 6DoF 위치/자세 추적을 게임에 사용할 수 있는가?
- 현재 상태: Galaxy Chrome에서 AR 실행, 위치 추적, hit-test, 숨기기, 이동 탐색, 발견 판정까지 동작 확인
- 다음 목표: SCAN 버튼을 MediaPipe의 주먹 -> 가위 -> 주먹 제스처로 교체

중요한 해석

현재 20초 스캔은 정밀한 3D mesh를 복원하는 SLAM map 생성이 아닙니다. WebXR hit-test로 실제 표면의 3D 좌표를 여러 개 수집해 "숨기기 후보 지점 집합"을 만드는 기능입니다.

1. 데모를 만든 목적

- Android Chrome에서 WebXR을 통해 ARCore 기반 6DoF 위치/자세 추적이 가능한지 확인
- 사용자가 실제로 걸어도 가상 인형이 현실 공간의 같은 위치에 남아 있는지 확인
- 바닥/책상 같은 실제 표면에서 숨기기 후보 위치를 만들 수 있는지 확인
- 실제 이동 + 화면 방향을 이용해 탐색/발견 게임 로직을 구성할 수 있는지 확인
- 추후 손동작, 보호색, occlusion, semantic mapping을 붙일 수 있는 구조인지 확인

2. 전체 동작 흐름



3. 현재 들어가 있는 주요 모듈

모듈	사용 기술	역할	상태
WebXR Session	WebXR immersive-ar	브라우저에서 AR 세션 시작	완료
6DoF Pose Tracking	WebXR + ARCore	스마트폰 3D 위치(x,y,z)와 회전 추적	완료
Rendering	Three.js	Reticle, Ninja, 3D 시각 요소 렌더링	완료
Surface Hit-Test	WebXR Hit Test	바닥/책상 등 실제 표면의 3D 좌표 획득	완료
Mapping Candidate Collector	자체 JS 로직	20초 동안 숨기기 후보 좌표 수집	완료
Hiding Spot Selector	자체 JS 로직	후보 중 적절한 위치를 랜덤 선택	완료
Camouflage Renderer	Three.js material	Ninja를 약 13% 투명도로 표시	기초
Hunt / Scan Logic	자체 JS 로직	거리와 시야각으로 발견 여부 판정	완료
Re-hide	자체 JS 로직	기존 후보를 재사용해 다시 숨기기	완료
Pose / Distance Logger	WebXR pose	이동경로, 최대 범위 표시	완료
Drift Check	자체 JS 로직	기준점 저장 후 복귀 위치/각도 오차 확인	완료
Gesture Trigger	MediaPipe 예경	주먹 -> 가위 -> 주먹으로 SCAN 실행	미구현

모듈 경계

ARCore/WebXR은 공간 추적과 표면 좌표를 제공하고, Three.js는 화면에 가상 오브젝트를 그립니다. 숨기기 위치 선택과 SCAN 판정은 프로젝트에서 직접 작성한 게임 로직입니다.

4. 모듈별 동작 설명

4.1 WebXR / ARCore 6DoF Tracking

단순 자이로 값만 사용하는 것이 아니라 WebXR을 통해 ARCore가 계산한 6DoF pose를 사용합니다. Translation은 x, y, z이며 회전 정보도 함께 얻습니다. 사용자가 실제로 2~3m 이동하면 좌표가 변하고, 이 덕분에 Ninja를 화면이 아니라 현실 공간의 한 위치에 고정할 수 있습니다.

4.2 Surface Hit-Test

카메라 중앙 방향으로 hit-test를 수행해 실제 표면과 만나는 3D 위치를 얻습니다. 바닥이나 책상을 비추면 해당 표면 위의 좌표를 후보로 사용할 수 있습니다.

4.3 20초 Mapping Candidate Collector

약 250ms 간격으로 표면 좌표를 확인하고, 이전 후보와 너무 가까운 점은 중복 저장하지 않습니다. 목적은 정밀 지도 복원이 아니라 "인형을 놓을 수 있는 실제 좌표 목록"을 만드는 것입니다.

4.4 Hiding Spot Selector

수집된 후보 중 한 점을 선택해 Ninja 위치로 사용합니다. 현재는 사용자와 너무 가깝지 않고, 처음부터 정면에 너무 잘 보이지 않으며, 탐색 가능한 거리의 후보를 조금 더 우선합니다.

4.5 Camouflage

현재 Ninja는 약 13% opacity로 렌더링됩니다. 즉 "반투명 위장"까지만 구현된 상태이며, 주변 영상의 색이나 texture를 분석해 동적으로 보호색을 만드는 단계는 아직 아닙니다.

4.6 Scan / Detection

사용자가 의심되는 방향을 화면 중앙 십자선에 맞춘 후 SCAN 버튼을 누릅니다. 기본 조건은 Ninja까지 거리 5m 이내, 카메라 정면과 Ninja 방향의 각도 12deg 이내입니다. 둘 다 만족하면 DETECTED!, 아니면 NO SIGNAL입니다.

4.7 Drift Check

기준점 저장 시 현재 pose를 기록하고, 이동 또는 회전 후 같은 위치/방향으로 돌아왔을 때 위치 오차(m)와 회전 오차(deg)를 계산합니다. 정밀 ground truth 측정이 아니라 실제 게임용 tracking 안정성을 빠르게 보는 진단 기능입니다.

5. 현재 실제 기기에서 확인된 기능

- Galaxy Chrome에서 WebXR AR 세션 실행
- 스마트폰의 3D 위치/회전 추적
- 실제 표면 hit-test 및 20초 후보 수집
- 후보 중 Ninja 위치 선택 및 반투명 렌더링
- 사용자가 실제로 이동하면서 다른 시점에서 Ninja 관찰
- 화면 중앙 조준 + SCAN, 거리/시야각 기반 발견 판정
- 같은 후보 집합에서 다시 숨기기
- 이동거리/최대 변위 표시, 기준점 복귀 오차 확인

6. 현재 한계

한계	의미
정밀 3D Map 아님	현재는 sparse한 표면 후보 좌표를 수집합니다. 물체의 전체 mesh나 방의 완전한 형상을 재구성하지 않습니다.
Semantic 정보 없음	현재 좌표가 책상인지 침대인지, 벽인지 등을 구분하지 않습니다.
손동작 미연결	주먹 -> 가위 -> 주먹은 아직 AR 화면과 통합되지 않았고 SCAN 버튼이 대신합니다.
진짜 보호색 미구현	현재는 단순 투명도 조절입니다. 주변 색/texture 기반 camouflage는 미구현입니다.
Occlusion 미구현	실제 책상/가구가 Ninja를 자연스럽게 가리는 depth/mesh 기반 처리는 아직 없습니다.
대규모 공간 미검증	150m x 30m 규모는 구역화, 장시간 tracking 안정성, 재위치화 전략이 추가로 필요합니다.

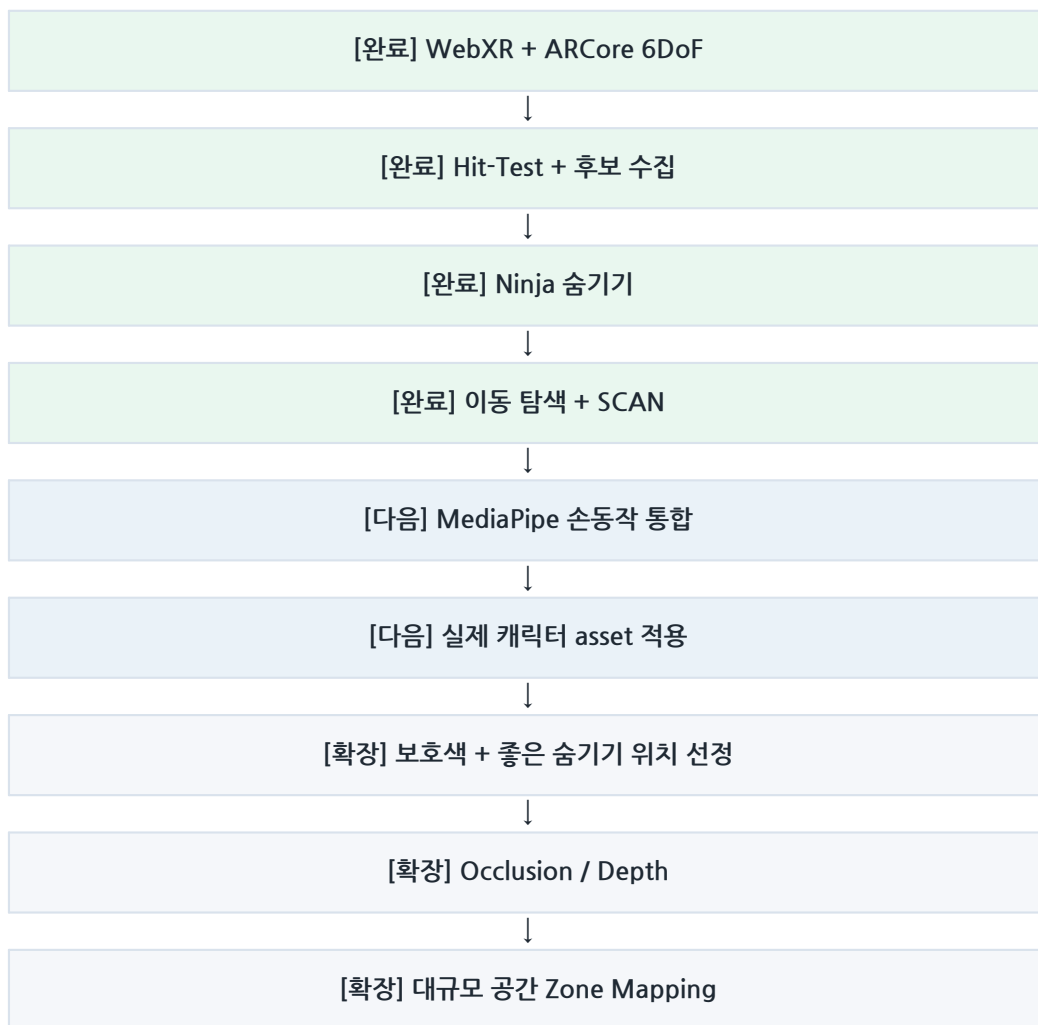
7. 추가 개발 가능한 기능

우선순위	기능	개발 내용
A-1	손동작 Trigger	MediaPipe Hands / Gesture Recognizer로 주먹 -> 가위 -> 주먹 상태 머신 구현. 현재 SCAN 버튼을 gesture event로 교체.
A-2	실제 캐릭터 asset	PNG billboard 또는 glTF/GLB 3D 모델을 적용하고 크기/방향을 조절.
A-3	난이도 조절	투명도, 발견 거리, 발견 시야각, 힌트 주기를 파라미터화.
B-1	주변 색 기반 보호색	카메라 영상의 target 주변 색/특징을 추출해 캐릭터 tint 또는 texture를 동적으로 조절.
B-2	좋은 숨기기 위치 선정	구석, 가구 근처, 시야에서 덜 노출된 위치에 높은 hiding score 부여.
B-3	Occlusion	WebXR Depth / ARCore Depth 계열을 검토해 실제 물체가 앞에 있으면 Ninja가 가려지도록 처리.
B-4	힌트 시스템	Hot/Cold, 거리별 진동/소리, 스캔 강도 같은 게임 피드백 추가.
C-1	Semantic Mapping	바닥/벽/책상/의자 등 의미 정보를 붙이고 숨기기 규칙과 연결.
C-2	대규모 Zone Mapping	긴 공간을 여러 Zone으로 나누고 Zone별 후보 및 재위치화 전략을 사용.
C-3	성능 평가	스캔 시간, 조명, texture, 이동 경로별 drift 및 탐색 성공률 비교.

추천 다음 단계

1) MediaPipe 손동작 통합 -> 2) 실제 캐릭터 asset -> 3) 보호색/숨기기 위치 품질 개선 -> 4) depth/occlusion -> 5) 대규모 공간 Zone Mapping 순서가 가장 현실적입니다.

8. 추천 개발 순서



9. 역할 분담 예시

역할	담당 기능
AR / Tracking	WebXR, ARCore pose, hit-test, anchor 안정성
Mapping / Game Logic	후보 수집, 숨기기 위치 선정, 거리/각도 판정
Gesture	MediaPipe, 주먹-가위-주먹 sequence detection
Rendering	캐릭터 asset, 투명도, 보호색, 시각 효과
Evaluation	drift, scan time, 성공률, device/환경별 성능 측정

발표에서 강조할 핵심

현재 프로토타입의 핵심 성과는 "웹 브라우저에서도 ARCore 기반 6DoF 추적과 실제 표면 좌표를 이용해, 사용자가 공간을 이동하면서 가상 물체를 탐색하는 게임 루프가 실제 기기에서 동작했다"는 점입니다.